

Level up your Git commits



```
git commit -m "feat: message"
```

feat - Introducing New Features

When you add an exciting new feature, use 'feat' commit type. It helps team members identify and track significant changes easily



```
git commit -m "fix: message"
```

fix – Fixing Bugs

Addressing a bug? Utilize 'fix' commit type to highlight that the commit resolves an issue or a problem in the codebase.



```
git commit -m "chore: message"
```

chore – Miscellaneous Changes

For non-feature/non-fix updates, like updating dependencies or performing routine tasks, use 'chore' commit type.



```
git commit -m "refactor: message"
```

refactor – Code Refactoring

When you improve the structure or readability of the code without adding features or fixing bugs, use 'refactor' commit type.



```
git commit -m "docs: message"
```

docs - Documentation Updates

Updating project documentation?
Whether it's the README or other relevant files, 'docs' commit type keeps the documentation up to date.



```
git commit -m "style: message"
```

style - Code Formatting

For changes that don't affect code behavior but enhance code style or formatting, like whitespace or missing semi-colons, use 'style' commit type.



```
git commit -m "test: message"
```

test - Testing Updates

When you add or modify tests to ensure code quality and reliability, use 'test' commit type to keep track of testing efforts.



```
git commit -m "perf: message"
```

perf – Performance Improvements

Did you optimize the code to boost performance? Use 'perf' commit type to communicate enhancements that make your code faster and more efficient.



```
git commit -m "ci: message"
```

ci – Continuous Integration

Changes related to continuous integration, such as configuring pipelines or adjusting build processes, should use the 'ci' commit type.



```
git commit -m "build: message"
```

build – Build System Updates

For changes impacting the build system or external dependencies, such as package updates or build script modifications, use 'build' commit type.



```
git commit -m "revert: message"
```

revert - Reverting Changes

Mistakes happen! If you need to undo a previous commit, use 'revert' commit type to roll back changes and restore the previous state.



Level up your Git commits

Start using them today!

~ Rishi Meena
Founder/CTO (errsole)

